



Arquitetura e Independência de Dados

Aula 10 · Bloco 3 — O SGBD na Prática

Três níveis, e o que cada um protege

Nesta aula

1. **Esquema x instância**, e por que a diferença é cara
2. A arquitetura em **três níveis**
3. Independência **física**
4. Independência **lógica**
5. **DDL, DML, DCL**
6. O **catálogo**

Esquema x instância

ESQUEMA (a forma)

USUARIO(matricula, nome,
categoria)

INSTÂNCIA (hoje às 14h32)

(2023101, 'Ana Souza', 'aluno')
(2023102, 'Bruno Lima', 'aluno')
(2024007, 'Célia Reis', 'professor')

O esquema é a **planta da casa**. A instância é a casa mobiliada num certo dia.

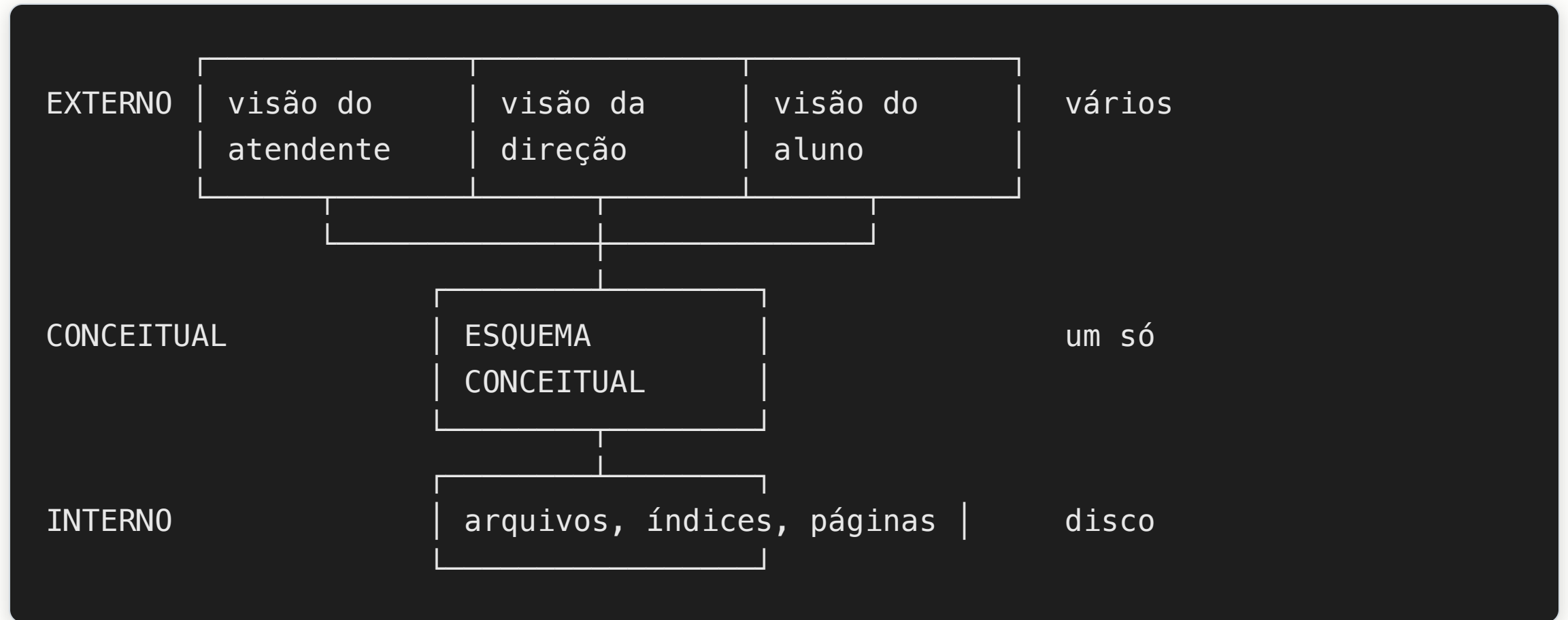
Mudar a planta é obra. Trocar os móveis é terça-feira.

Erro de esquema x erro de instância

Dado errado se corrige com um comando.

Esquema errado se corrige migrando
todos os dados existentes,
reescrevendo todo programa que o usa,
e explicando à gerência por que
o sistema vai ficar fora do ar no sábado.

Os três níveis



Os três, sobre o mesmo dado

Nível	O que se diz
Externo	"Empréstimos vencidos: nome, título, dias de atraso"
Conceitual	<code>EMPRESTIMO(id, matricula, tombo, ...)</code> com FKs
Interno	páginas de 8 KB, índice em <code>id</code> , índice em <code>data_prevista</code>

O **conceitual** é exatamente o que você construiu nos Blocos 1 e 2.

Independência física e lógica

Física — mudar o **interno** sem mexer no resto. Criar índice, trocar disco. Você recebe de graça, porque a ligação é **por valor** (Aula 03).

Lógica — mudar o **conceitual** sem mexer nas visões. Acrescentar coluna, dividir tabela.

Mudança	Protege
criar índice · trocar disco	física
acrescentar coluna · dividir tabela	lógica
renomear uma coluna	nenhuma



A independência lógica é conquistada

O programa que pede "todas as colunas"
e confia na ordem delas
perde a independência na primeira alteração.

O que pede as colunas **pelo nome** sobrevive.

Escrever os nomes é uma decisão de arquitetura
disfarçada de estilo.

As três famílias de comandos

```
CREATE TABLE usuario (...);           -- DDL: cria a forma
INSERT INTO usuario VALUES (...);     -- DML: põe conteúdo
SELECT * FROM usuario;                 -- DML: consulta
GRANT SELECT ON usuario TO atendente;  -- DCL: autoriza
DROP TABLE usuario;                   -- DDL: destrói a forma
```

DDL mexe na forma · **DML** mexe no conteúdo · **DCL** controla quem pode



DELETE e **DROP** não são sinônimos

```
DELETE FROM usuario;
```

apaga as linhas e deixa a tabela **de pé**.

```
DROP TABLE usuario;
```

apaga a tabela inteira —
estrutura, chaves, restrições e dados.

O primeiro se desfaz numa transação.
O segundo, não.

O catálogo: o banco que descreve o banco

```
SELECT table_name, column_name, data_type  
FROM information_schema.columns  
WHERE table_schema = 'public'  
ORDER BY table_name, ordinal_position;
```

Isso devolve **o esquema conceitual do seu banco, lido do próprio banco.**

É a definição de **metadado**: dado sobre dado.

Um sistema autodescritivo

Um arquivo `.csv` não sabe dizer
que tipo tem a terceira coluna.

Um banco sabe —
e pode **agir** sobre isso,
recusando o que não couber.

É uma das diferenças de fundo
entre um banco e um punhado de arquivos.

Exercícios da aula

Na pasta `aula-10/` :

1. `ex01.md` — esquema ou instância? Seis afirmações;
2. `ex02.md` — três visões externas da Biblioteca, e o que difere entre elas;
3. `ex03.md` — que independência protege cada mudança? Uma delas: nenhuma;
4. `ex04.md` — DDL, DML ou DCL? Dez comandos;
5. **Desafio**  `ex05.md` — dividir `USUARIO` em duas tabelas sem quebrar os programas.

Próxima aula

Aula 11 — PostgreSQL na prática

Chega de papel.

Instalar, criar o banco
e olhar o catálogo por dentro.